

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE

◆ Analisi del codice **LogLeggiScrivi**

1. Scopo generale

Il file definisce una classe **LogLeggiScrivi** che fornisce un semplice sistema di logging per un'applicazione Windows Forms.

- **Scrittura:** aggiunge eventi, eccezioni o messaggi generici a un file di log (**ItalPascal_Log.txt**) situato nella cartella di esecuzione dell'applicazione.
- **Lettura:** apre il file di log con l'applicazione predefinita del sistema operativo.
- **Pulizia:** mantiene solo gli ultimi *N* eventi (default 5) eliminando il resto.

La classe è **NotInheritable** (finale) e tutti i metodi sono **Shared** , quindi non è necessario creare un'istanza per usarla.

2. Componenti chiave

Metodo	Funzione	Parametri	Note
ApriLog (Optional NomeFileLog As String = NomeFileLogDefault)	Apre il file di log con l'applicazione di default.	NomeFileLog (opzionale)	Restituisce il file di log. Falso se non trovato.
ScriviLog (Procedura As String, ex As Exception)	Scrive un'eccezione dettagliata.	Procedura (nome della routine), ex (eccezione)	Aggiunge l'eccezione al file di log. Veramente se l'eccezione è diversa da Nothing .
ClearLog (Optional NumeroEventiDaMantenere As Integer = 5)	Mantiene solo gli ultimi <i>N</i> eventi.	NumeroEventiDaMantenere	Gli eventi precedenti vengono eliminati. Se non viene specificato, il default è 5.

Metodo	Funzione	Parametri	Note
<code>ScriviLogMsg(Messaggio As String)</code>	Scrive un messaggio generico.	<code>Messaggio</code>	Utilizzato non
Costanti	<code>NomeFileLogDefault</code>	Stringa con il nome del file di log.	Il va Ita

Struttura del file di log

Ogni evento è delimitato da una riga di 80 caratteri (= o -).

Un evento di eccezione contiene:

```

1  Data      : dd/MM/yyyy HH:mm:ss
2  Procedura : <nome>
3  Messaggio : <ex.Message>
4
5  Dettagli eccezione:
6  <ex.ToString()>
```

Un evento di messaggio contiene:

```

1  Data      : dd/MM/yyyy HH:mm:ss
2  Evento    : <Messaggio>
```

3. Flusso di esecuzione

3.1 Apertura del log (`ApriLog`)

- Costruzione del percorso:** `Path.Combine(AppDomain.CurrentDomain.BaseDirectory, NomeFileLog)`.
- Verifica esistenza:** se il file non esiste, mostra un `MessageBox` di avviso e restituisce `False`.
- Apertura:** `Process.Start` con `UseShellExecute = True` apre il file con l'applicazione predefinita (es. Notepad).
- Gestione eccezioni:** eventuali errori mostrano un `MessageBox` di errore e restituiscono `False`.

3.2 Scrittura di un'eccezione (`ScriviLog`)

- Percorso:** come sopra, ma sempre con `NomeFileLogDefault`.
- StreamWriter:** aperto in modalità append (`True`).
- Scrittura:** header, data, procedura, messaggio, dettagli dell'eccezione.
- Try/Catch:** in caso di errore di scrittura, l'eccezione viene ignorata (commento "Evita errori durante il logging").

3.3 Scrittura di un messaggio (`ScriviLogMsg`)

1. **Percorso:** come sopra.
2. **StreamWriter:** modalità append.
3. **Scrittura:** header, data, evento.
4. **Try/Catch:** idem a `ScriviLog` .

3.4 Pulizia del log (`ClearLog`)

1. **Percorso:** come sopra.
 2. **Legge tutte le righe** con `File.ReadAllLines` .
 3. **Separazione degli eventi:**
 - Quando una riga inizia con `===` o `---` , si considera l'inizio di un nuovo evento.
 - Le righe vengono raccolte in una lista di liste (`eventi`).
 4. **Selezione degli ultimi N eventi:** `eventi.Skip(eventi.Count - NumeroEventiDaMantenere)` .
 5. **Scrittura:** apre un nuovo `StreamWriter` in modalità overwrite (`False`) e scrive solo gli eventi selezionati.
 6. **Gestione eccezioni:** mostra un `MessageBox` in caso di errore.
-

4. Possibili utilizzi

Il frammento mostra come **scrivere messaggi di log** in un'applicazione VB.NET. Il logger è fornito da una classe chiamata `LogLeggiScrivi` con il metodo `ScriviLogMsg(String)` . I messaggi registrati possono essere usati per tracciare l'avvio dell'applicazione, operazioni di salvataggio, cancellazioni o per diagnosticare eventuali errori.

```
1      ' Esempi di utilizzo:
2      LogLeggiScrivi.ScriviLogMsg("Applicazione avviata.")
3      LogLeggiScrivi.ScriviLogMsg("Anagrafica cliente salvata
correttamente.")
4      LogLeggiScrivi.ScriviLogMsg($"Salvati {dgvClienti.Rows.Count}
record.")
5      LogLeggiScrivi.ScriviLogMsg($"Cliente ID {IDCliente} eliminato.")
6
7
8      Dim Utente As String =
FatDataGrid.CurrentRow.Cells("5").Value.ToString()
9      Dim NumeroFattura As String =
FatDataGrid.CurrentRow.Cells("3").Value.ToString()
10     LogLeggiScrivi.ScriviLogMsg( $"Fattura {NumeroFattura}"& "del Sig.
${Utente}" & "salvata correttamente.")
```

1. Costruzione e log del messaggio di fattura

```
1 LogLeggiScrivi.ScriviLogMsg( $"Fattura {NumeroFattura}"& "del Sig. "
  & $"{Utente}" & "salvata correttamente.")
```

- Si tenta di concatenare una stringa interpolata con altre parti usando l'operatore `&`.
- Il risultato finale dovrebbe essere:

```
Fattura 1234 del Sig. Mario Rossi salvata correttamente.
```

5. Come usarlo

```
1 ' Apri il log (mostra il file)
2 LogLeggiScrivi.ApriLog()
3
4 ' Scrivi un messaggio semplice
5 LogLeggiScrivi.ScriviLogMsg("Applicazione avviata.")
6
7 ' Scrivi un'eccezione
8 Try
9     ' codice che può generare eccezione
10 Catch ex As Exception
11     LogLeggiScrivi.ScriviLog(NumberOf(CaricaDati), ex)
12 End Try
13
14 ' Pulisci il log mantenendo solo gli ultimi 10 eventi
15 LogLeggiScrivi.ClearLog(10)
```

◆ Conclusione

Il codice fornisce una soluzione di logging di base, facile da integrare in progetti Windows Forms.

Con alcune piccole modifiche (gestione degli errori, thread-safety, delimitatori più robusti) può diventare un componente più affidabile e scalabile.